



FACULTAD DE INGENIERIA Y CIENCIAS EXACTAS
Departamento de Tecnología Informática Nivel Inicial

PROGRAMACIÓN I

ALGORITMOS Y ESTRUCTURAS DE DATOS I

Guía de Trabajos Prácticos

v26.03.01

Bibliografía sugerida:

Básica

- Joyanes Aguilar, Luis: Fundamentos de Programación. Editorial Mc. Graw-Hill. ISBN 978-84-481-6111-8. EAN 9788448161118.

Complementaria

- **Pilgrin, Mark:** Inmersión en Python 3 [en línea] Traducido por José Miguel González Aguilera. Disponible gratuitamente (bajo licencia Creative Commons 3.0). [Revisado 22/02/26]
<http://www.imgaguilera.com/libro/python/traduccion/C3%B3n/latex/2016/08/19/inmersion-python.html>
- **Marzal Varó, Andrés - Gracia Luengo, Isabel, García Sevilla, Pedro:** Introducción a la programación con Python 3 [En línea] Disponible gratuitamente bajo licencia Creative Commons. [Revisado: 22/02/26]
<https://repo.umc.cl/items/affd609a-bf21-41ba-80d4-f1a22a4be0d1/full>

Autores:

Galati Verónica – Thompson Ricardo – Escalante Ramiro

Trabajo Práctico 1: Funciones

1. Desarrollar una función que reciba tres números enteros positivos y devuelva el mayor de los tres, sólo si éste es único (es decir el mayor estricto). Devolver -1 en caso de no haber ninguno. No utilizar operadores lógicos (and, or, not). Desarrollar también un programa para ingresar los tres valores, invocar a la función y mostrar el máximo hallado, o un mensaje informativo si éste no existe.
2. Desarrollar una función que reciba tres números enteros positivos correspondientes al día, mes, año de una fecha y verifique si corresponden a una fecha válida. Debe tenerse en cuenta la cantidad de días de cada mes, incluyendo los años bisiestos. Devolver True o False según la fecha sea correcta o no. Realizar también un programa para verificar el comportamiento de la función.
3. Una persona desea llevar el control de los gastos realizados al viajar en el subterráneo dentro de un mes. Sabiendo que dicho medio de transporte utiliza un esquema de tarifas decrecientes (detalladas en la tabla de abajo) se solicita desarrollar una función que reciba como parámetros la cantidad de viajes realizados en un determinado mes y el valor del pasaje, y devuelva el total gastado en viajes. Realizar también un programa para verificar el comportamiento de la función.

Cantidad de viajes	Valor del pasaje
1 a 20	Averiguar en Internet el valor actualizado
21 a 30	20% de descuento sobre tarifa máxima
31 a 40	30% de descuento sobre tarifa máxima
Más de 40	40% de descuento sobre tarifa máxima

4. Un comercio de electrodomésticos necesita para su línea de cajas un programa que le indique al cajero el cambio que debe entregarle al cliente. Para eso se ingresan dos números enteros, correspondientes al total de la compra y al dinero recibido. Informar cuántos billetes de cada denominación deben ser entregados como vuelto, de tal forma que se minimice la cantidad de billetes. Considerar que existen billetes de \$5000, \$1000, \$500, \$200, \$100, \$50 y \$10. Emitir un mensaje de error si el dinero recibido fuera insuficiente o si el cambio no pudiera entregarse debido a falta de billetes con denominaciones adecuadas. Ejemplo: Si la compra es de \$3170 y se abona con \$5000, el vuelto debe contener 1 billete de \$1000, 1 billete de \$500, 1 billete de \$200, 1 billete de \$100 y 3 billetes de \$10.
5. Escribir funciones lambda para:
 - a. Informar si un número es oblongo. Se dice que un número es oblongo cuando se puede obtener multiplicando dos números naturales consecutivos. Por ejemplo 6 es oblongo porque resulta de multiplicar $2 * 3$.
 - b. Informar si un número es triangular. Un número se define como triangular si puede expresarse como la suma de un grupo de números naturales consecutivos comenzando desde 1. Por ejemplo 10 es un número triangular porque se obtiene sumando $1+2+3+4$.

Ambas funciones lambda reciben como único parámetro el número a evaluar y devuelven True o False. No se permite utilizar ayudas externas a las mismas.

6. Desarrollar una función que reciba como parámetros dos números enteros positivos y devuelva como valor de retorno el número que resulte de concatenar ambos parámetros. Por ejemplo, si recibe 1234 y 567 debe devolver 1234567. No se permite utilizar facilidades de Python no vistas en clase.
7. Escribir una función diasiguiente(dia, mes año) que reciba como parámetro una fecha cualquiera expresada por tres enteros y calcule y devuelva otros tres enteros correspondientes el día siguiente al dado. Utilizando esta función sin modificaciones ni agregados, desarrollar programas que permitan:
 - a. Sumar N días a una fecha.
 - b. Calcular la cantidad de días existentes entre dos fechas cualesquiera.
8. La siguiente función permite averiguar el día de la semana para una fecha determinada. La fecha se suministra en forma de tres parámetros enteros y la función devuelve 0 para domingo, 1 para lunes, 2 para martes, etc. Escribir un programa para imprimir por pantalla el calendario de un mes completo, correspondiente a un mes y año cualquiera basándose en la función suministrada. Considerar que la semana comienza en domingo.

```
def diadelasemana(dia,mes,año):  
    if mes < 3:  
        mes = mes + 10  
        año = año - 1  
    else:  
        mes = mes - 2  
    siglo = año // 100  
    año2 = año % 100  
    diasem = ((26*mes-2)//10)+dia+año2+(año2//4)+(siglo//4)-(2*siglo)%7  
    if diasem < 0:  
        diasem = diasem + 7  
    return diasem
```

9. Resolver el siguiente problema utilizando funciones:

Un productor frutihortícola desea contabilizar sus cajones de naranjas según el peso para poder cargar los camiones de reparto. La empresa cuenta con N camiones, y cada uno puede transportar hasta media tonelada (500 kilogramos). En un cajón caben 100 naranjas con un peso de entre 200 y 300 gramos cada una. Si el peso de alguna naranja se encuentra fuera del rango indicado se la clasifica para procesar como jugo. Desarrollar un programa para ingresar la cantidad de naranjas cosechadas e informar cuántos cajones se pueden llenar, cuántas naranjas son para jugo y si hay algún sobrante de naranjas que deba considerarse para el siguiente reparto. Simular el peso de cada unidad generando un número entero al azar entre 150 y 350.

Además, se desea saber cuántos camiones se necesitan para transportar la cosecha, considerando que la ocupación del camión no debe ser inferior al 80%; en caso contrario el camión no será despachado por su alto costo.

Trabajo Práctico 2: Listas

1. Desarrollar cada una de las siguientes funciones y escribir un programa que permita verificar su funcionamiento imprimiendo la lista luego de invocar a cada función:
 - a. Cargar una lista con números al azar de cuatro dígitos. La cantidad de elementos también será un número al azar de dos dígitos.
 - b. Calcular y devolver el producto de todos los elementos de la lista anterior.
 - c. Eliminar todas las apariciones de un valor en la lista anterior. El valor a eliminar se ingresa desde el teclado y la función lo recibe como parámetro. No utilizar listas auxiliares.
 - d. Determinar si el contenido de una lista cualquiera es capicúa, sin usar listas auxiliares. Un ejemplo de lista capicúa es [50, 17, 91, 17, 50].
2. Escribir funciones para:
 - a. Generar una lista de N números aleatorios del 1 al 100. El valor de N se ingresa a través del teclado.
 - b. Recibir una lista como parámetro y devolver True si la misma contiene algún elemento repetido. La función no debe modificar la lista.
 - c. Recibir una lista como parámetro y devolver una nueva lista con los elementos únicos de la lista original, sin importar el orden.Combinar estas tres funciones en un mismo programa.
3. Crear una lista con los cuadrados de los números entre 1 y N (ambos incluidos), donde N se ingresa desde el teclado. Luego se solicita imprimir los últimos 10 valores de la lista.
4. Eliminar de una lista de números enteros aquellos valores que se encuentren en una segunda lista. Imprimir la lista original, la lista de valores a eliminar y la lista resultante. La función debe modificar la lista original sin crear una copia modificada.
5. Escribir una función que reciba una lista como parámetro y devuelva True si la lista está ordenada en forma ascendente o False en caso contrario. Por ejemplo, ordenada([1, 2, 3]) retorna True y ordenada(['b', 'a']) retorna False. Desarrollar además un programa para verificar el comportamiento de la función.
6. Escribir una función que reciba una lista de números enteros como parámetro y la normalice, es decir que todos sus elementos deben sumar 1.0, respetando las proporciones relativas que cada elemento tiene en la lista original. Desarrollar también un programa que permita verificar el comportamiento de la función. Por ejemplo, normalizar([1, 1, 2]) debe devolver [0.25, 0.25, 0.50].
7. Intercalar los elementos de una lista entre los elementos de otra. La intercalación deberá realizarse exclusivamente mediante la técnica de rebanadas y no se creará una lista nueva sino que se modificará la primera. Por ejemplo, si lista1 = [8, 1, 3] y lista2 = [5, 9, 7], lista1 deberá quedar como [8, 5, 1, 9, 3, 7]. Las listas pueden tener distintas longitudes.

8. Utilizar la técnica de listas por comprensión para construir una lista con todos los números impares comprendidos entre 100 y 200.
9. Generar e imprimir una lista por comprensión entre A y B con los múltiplos de 7 que no sean múltiplos de 5. A y B se ingresan desde el teclado.
10. Generar una lista con números al azar entre 1 y 100 y crear una nueva lista con los elementos de la primera que sean impares. El proceso deberá realizarse utilizando la función `filter()`. Imprimir las dos listas por pantalla.
11. Resolver el siguiente problema, diseñando las funciones a utilizar:

Una clínica necesita un programa para atender a sus pacientes. Cada paciente que ingresa se anuncia en la recepción indicando su número de afiliado (número entero de 4 dígitos) y además indica si viene por una urgencia (ingresando un 0) o con turno (ingresando un 1). Para finalizar se ingresa -1 como número de socio. Luego se solicita:

- a. Mostrar un listado de los pacientes atendidos por urgencia y un listado de los pacientes atendidos por turno en el orden que llegaron a la clínica.
- b. Realizar la búsqueda de un número de afiliado e informar cuántas veces fue atendido por turno y cuántas por urgencia. Repetir esta búsqueda hasta que se ingrese -1 como número de afiliado.

12. Resolver el siguiente problema, utilizando funciones:

Se desea llevar un registro de los socios que visitan un club cada día. Para ello, se ingresa el número de socio de cinco dígitos hasta ingresar un cero como fin de carga. Se solicita:

- a. Informar para cada socio, cuántas veces ingresó al club. Cada socio debe aparecer una sola vez en el informe.
- b. Solicitar un número de socio que se dio de baja del club y eliminar todos sus ingresos. Mostrar los registros de entrada al club antes y después de eliminarlo. Informar cuántos ingresos se eliminaron.

Trabajo Práctico 3: Matrices

1. Desarrollar cada una de las siguientes funciones y escribir un programa que permita verificar su funcionamiento, imprimiendo la matriz luego de invocar a cada función:
 - a. Cargar números enteros en una matriz de $N \times N$, ingresando los datos desde teclado.
 - b. Ordenar en forma ascendente cada una de las filas de la matriz.
 - c. Intercambiar dos filas, cuyos números se reciben como parámetro.
 - d. Intercambiar dos columnas dadas, cuyos números se reciben como parámetro.
 - e. Trasponer la matriz sobre si misma. (intercambiar cada elemento A_{ij} por A_{ji})
 - f. Calcular el promedio de los elementos de una fila, cuyo número se recibe como parámetro.
 - g. Calcular el porcentaje de elementos con valor impar en una columna, cuyo número se recibe como parámetro.
 - h. Determinar si la matriz es simétrica con respecto a su diagonal principal.
 - i. Determinar si la matriz es simétrica con respecto a su diagonal secundaria.
 - j. Determinar qué columnas de la matriz son palíndromos (capicúas), devolviendo una lista con los números de las mismas.

NOTA: El valor de N debe leerse por teclado. Las funciones deben servir cualquiera sea el valor ingresado.

2. Las siguientes matrices responden distintos patrones de relleno. Desarrollar funciones que generen cada una de ellas sin intervención humana y escribir un programa que las invoque e imprima por pantalla. El tamaño de las matrices debe establecerse como $N \times N$, donde N se ingresa a través del teclado.

a:

1	0	0	0
0	3	0	0
0	0	5	0
0	0	0	7

b:

0	0	0	27
0	0	9	0
0	3	0	0
1	0	0	0

c:

4	0	0	0
3	3	0	0
2	2	2	0
1	1	1	1

d:

8	8	8	8
4	4	4	4
2	2	2	2
1	1	1	1

e:

0	1	0	2
3	0	4	0
0	5	0	6
7	0	8	0

f:

0	0	0	1
0	0	3	2
0	6	5	4
10	9	8	7

g:

1	2	3	4
12	13	14	5
11	16	15	6

h:

1	2	4	7
3	5	8	11
6	9	12	14

i:

1	2	6	7
3	5	8	13
4	9	12	14

10	9	8	7
----	---	---	---

10	13	15	16
----	----	----	----

10	11	15	16
----	----	----	----

- Desarrollar un programa para rellenar una matriz de $N \times N$ con números enteros al azar comprendidos en el intervalo $[0, N^2)$, de tal forma que ningún número se repita. Imprimir la matriz por pantalla.
- Una fábrica de bicicletas guarda en una matriz la cantidad de unidades producidas en cada una de sus plantas durante una semana. De este modo, cada columna representa el día de la semana y cada fila a una de sus fábricas. Ejemplo:

		(Lunes)	(Martes)	(Miércoles)	(Jueves)	(Viernes)	(Sábado)
		0	1	2	3	4	5
(Fábrica 1)	0	23	150	20	120	25	150
(Fábrica 2)	1	40	75	80	0	80	35
(...)		...	...	...	...	...	...
(Fábrica n)	...	80	80	80	80	80	80

Se solicita:

- Crear una matriz con datos generados al azar para N fábricas durante una semana, considerando que la capacidad máxima de fabricación es de 150 unidades por día y puede suceder que en ciertos días no se fabrique ninguna.
 - Mostrar la cantidad total de bicicletas fabricadas por cada fábrica.
 - Cuál es la fábrica que más produjo en un solo día (detallar día y fábrica).
 - Cuál es el día más productivo, considerando todas las fábricas combinadas.
 - Crear una lista por comprensión que contenga la menor cantidad fabricada por cada fábrica.
- Desarrollar un programa que permita realizar reservas en una sala de cine de N filas con M butacas por cada fila. Desarrollar las siguientes funciones y utilizarlas en un mismo programa:

mostrar_butacas: Mostrará por pantalla el estado de cada una de las butacas del cine. Esta función deberá ser invocada antes de que se realice la reserva, y se volverá a invocar luego de la misma con los estados actualizados.

reservar: Deberá recibir una matriz y la butaca seleccionada, y actualizará la sala en caso de estar disponible dicha butaca. La función devolverá **True/False** si logró o no reservar la butaca.

cargar_sala: Recibirá una matriz como parámetro y la cargará con valores aleatorios para simular una sala con butacas ya reservadas.

butacas_libres: Recibirá como parámetro la matriz y retornará cuántas butacas desocupadas hay en la sala.

butacas_contiguas: Buscará la secuencia más larga de butacas libres contiguas en una misma fila y devolverá las coordenadas de inicio de la misma.

Trabajo Práctico 4: Cadenas de caracteres

1. Desarrollar una función que determine si una cadena de caracteres es capicúa, sin utilizar cadenas auxiliares ni rebanadas. Escribir además un programa que permita verificar su funcionamiento.
2. Leer una cadena de caracteres e imprimirla centrada en pantalla. Suponer que la misma tiene 80 columnas.
3. Los números de claves de dos cajas fuertes están intercalados dentro de un número entero llamado "clave maestra", cuya longitud no se conoce. Realizar un programa para obtener ambas claves, donde la primera se construye con los dígitos ubicados en posiciones impares de la clave maestra y la segunda con los dígitos ubicados en posiciones pares. Los dígitos se numeran desde la izquierda. Ejemplo: Si clave maestra fuera 18293, la clave 1 sería 123 y la clave 2 sería 89.
4. Escribir una función que reciba como parámetro un número entero entre 0 y 3999 y lo convierta en un número romano, devolviéndolo en una cadena de caracteres. ¿En qué cambiaría la función si el rango de valores fuese diferente?
5. Escribir una función `filtrar_palabras()` que reciba una cadena de caracteres conteniendo una frase y un entero `N`, y devuelva otra cadena con las palabras que tengan `N` o más caracteres de la cadena original. Escribir también un programa para verificar el comportamiento de la misma. Hacer tres versiones de la función, para cada uno de los siguientes casos:
 - a. Utilizando sólo ciclos normales
 - b. Utilizando listas por comprensión
 - c. Utilizando la función `filter`
6. Desarrollar una función que extraiga una subcadena de una cadena de caracteres, indicando la posición y la cantidad de caracteres deseada. Devolver la subcadena como valor de retorno. Escribir también un programa para verificar el comportamiento de la misma. Ejemplo, dada la cadena "El número de teléfono es 4356-7890" extraer la subcadena que comienza en la posición 25 y tiene 9 caracteres, resultando la subcadena "4356-7890". Escribir una función para cada uno de los siguientes casos:
 - a. Utilizando rebanadas
 - b. Sin utilizar rebanadas
7. Escribir una función para eliminar una subcadena de una cadena de caracteres, a partir de una posición y cantidad de caracteres dadas, devolviendo la cadena resultante. Escribir también un programa para verificar el comportamiento de la misma. Escribir una función para cada uno de los siguientes casos:
 - a. Utilizando rebanadas
 - b. Sin utilizar rebanadas
8. Desarrollar una función que devuelva una subcadena con los últimos `N` caracteres de una cadena dada. La cadena y el valor de `N` se pasan como parámetros.

9. Escribir una función que reciba como parámetro una cadena de caracteres en la que las palabras se encuentran separadas por uno o más espacios. Devolver otra cadena con las palabras ordenadas según su longitud, dejando un espacio entre cada una. Los signos de puntuación no deben ser tenidos en cuenta al medir la longitud de las palabras, pero deberán conservarse en la cadena final.
10. Desarrollar una función para reemplazar todas las apariciones de una palabra por otra en una cadena de caracteres y devolver la cadena obtenida y un entero con la cantidad de reemplazos realizados. Tener en cuenta que sólo deben reemplazarse palabras completas, y no fragmentos de palabras. Escribir también un programa para verificar el comportamiento de la función.
11. Escribir un programa que cuente cuántas veces se encuentra una subcadena dentro de otra cadena, sin diferenciar mayúsculas y minúsculas. Tener en cuenta que los caracteres de la subcadena no necesariamente deben estar en forma consecutiva dentro de la cadena, pero sí respetando el orden de los mismos. Ejemplo:

Cadena:

Platero es pequeño, peludo, suave; tan blando por fuera, que se diría todo de algodón, que no lleva huesos. Sólo los espejos de azabache de sus ojos son duros cual dos escarabajos de cristal negro.

Sub-cadena: UADE

Cantidad encontrada: 4 (Las coincidencias están resaltadas en la cadena siguiente)

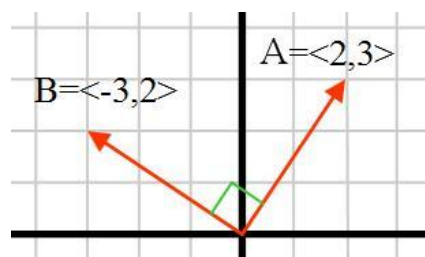
Platero es pequeño, peludo, suave; tan blando por fuera, que se diría todo de algodón, que no lleva huesos. Sólo los espejos de azabache de sus ojos son duros cual dos escarabajos de cristal negro.

12. Escribir un programa para crear una lista por comprensión con los naipes de la baraja española. La lista debe contener cadenas de caracteres. Ejemplo: ["1 Oros", "2 Oros"...]. Imprimir la lista por pantalla.
13. Muchas aplicaciones financieras requieren que los números sean expresados también en letras. Por ejemplo, el número 2153 puede escribirse como "dos mil ciento cincuenta y tres". Escribir un programa que utilice una función para convertir un número entero entre 0 y 1 billón (1.000.000.000.000) a letras. ¿En qué cambiaría la función si también aceptara números negativos? ¿Y números con decimales?
14. Se solicita crear un programa para leer direcciones de correo electrónico y verificar si representan una dirección válida. Por ejemplo usuario@dominio.com.ar. Para que una dirección sea considerada válida el nombre de usuario debe poseer solamente caracteres alfanuméricos, la dirección contener un solo carácter @, el dominio debe tener al menos un carácter y tiene que finalizar con .com o .com.ar.

Repetir el proceso de validación hasta ingresar una cadena vacía. Al finalizar mostrar un listado de todos los dominios, sin repetirlos y ordenados alfabéticamente, recordando que las direcciones de mail no distinguen mayúsculas ni minúsculas.

Trabajo Práctico 5: Tuplas, conjuntos y diccionarios

1. Desarrollar las siguientes funciones utilizando tuplas para representar fechas y horarios, y luego escribir un programa que las vincule:
 - a. Ingresar una fecha desde el teclado, verificando que corresponda a una fecha válida.
 - b. Sumar N días a una fecha.
 - c. Ingresar un horario desde teclado, verificando que sea correcto.
 - d. Calcular la diferencia entre dos horarios. Si el primer horario fuera mayor al segundo se considerará que el primero corresponde al día anterior. En ningún caso la diferencia en horas puede superar las 24 horas.
2. Escribir una función que reciba como parámetro una tupla conteniendo una fecha (día,mes,año) y devuelva una cadena de caracteres con la misma fecha expresada en formato extendido. La función debe contemplarse que el año se ingrese en dos dígitos, los que serán interpretados según un año de corte definido dentro del programa. Cualquier año mayor que éste se considerará del siglo pasado. Por ejemplo, si el año de corte fuera 30, la función devuelve "12 de Octubre de 2030" para (12,10,30). Pero si la tupla fuera (25, 12, 31) devolverá "25 de Diciembre de 1931". Si el año se ingresa en cuatro dígitos el año de corte no será tenido en cuenta. Escribir también un programa para ingresar los datos, invocar a la función y mostrar el resultado.
3. Desarrollar un programa que utilice una función que reciba como parámetro una cadena de caracteres conteniendo una dirección de correo electrónico y devuelva una tupla con las distintas partes que componen dicha dirección. Ejemplo: alguien@uade.edu.ar -> (alguien, uade, edu, ar). La función debe detectar formatos de fecha inválidos y devolver una tupla vacía.
4. Escribir una función que indique si dos fichas de dominó encajan o no. Las fichas son recibidas en dos tuplas, por ejemplo: (3, 4) y (5, 4). La función devuelve True o False. Escribir también un programa para verificar su comportamiento. Considerar el uso de conjuntos para resolver este ejercicio.
5. En geometría un vector es un segmento de recta orientado que va desde un punto A hasta un punto B. Los vectores en el plano se representan mediante un par ordenado de números reales (x, y) llamados *componentes*. Para representarlos basta con unir el origen de coordenadas con el punto indicado en sus componentes:



Dos vectores son *ortogonales* cuando son perpendiculares entre sí. Para determinarlo basta calcular su producto escalar y verificar si es igual a 0. Ejemplo:

$A = (2,3)$ y $B = (-3,2) \Rightarrow 2 * (-3) + 3 * 2 = -6 + 6 = 0 \Rightarrow$ Son ortogonales

Escribir una función que reciba dos vectores en forma de tuplas y devuelva un valor de verdad indicando si son ortogonales o no. Desarrollar también un programa que permita verificar el comportamiento de la función.

6. Ingresar una frase desde el teclado y usar un conjunto para eliminar las palabras repetidas, dejando un solo ejemplar de cada una. Finalmente mostrar las palabras ordenadas según su longitud. Los signos de puntuación no deben afectar el proceso.
7. Definir un conjunto con números enteros entre 0 y 9. Luego solicitar valores al usuario y eliminarlos del conjunto mediante el método `remove`, mostrando el contenido del conjunto luego de cada eliminación. Finalizar el proceso al ingresar -1. Utilizar manejo de excepciones para evitar errores al intentar quitar elementos inexistentes.
8. Generar e imprimir un diccionario donde las claves sean números enteros entre 1 y 20 (ambos incluidos) y los valores asociados sean el cuadrado de las claves.
9. Escribir un programa que permita ingresar un número entero N y genere un diccionario por comprensión con la tabla de multiplicar de N del 1 al 12. Mostrar la tabla de multiplicar con el formato apropiado.
10. Desarrollar una función `eliminarclaves()` que reciba como parámetros un diccionario y una lista de claves. La función debe eliminar del diccionario todas las claves contenidas en la lista, devolviendo el diccionario modificado y un número entero que represente la cantidad de claves eliminadas. Desarrollar también un programa para verificar su comportamiento.
11. Crear una función `contarvocales()`, que reciba una palabra y cuente cuántas vocales contiene, identificando la cantidad de cada una. Devolver un diccionario con los resultados. Luego desarrollar un programa para leer una frase e invocar a la función por cada palabra que contenga la misma. Imprimir las palabras y la cantidad de vocales hallada.
12. Una librería almacena su lista de precios en un diccionario. Diseñar un programa para crearlo, incrementar los precios de los cuadernos en un 15%, imprimir un listado con todos los elementos de la lista de precios e indicar cuál es el ítem más costoso que venden en el comercio.
13. Escribir una función `buscarclave()` que reciba como parámetros un diccionario y un valor, y devuelva una lista de claves que apunten ("mapeen") a ese valor en el diccionario. Comprobar el comportamiento de la función mediante un programa apropiado.

Trabajo Práctico 6: Manejo de excepciones

1. Desarrollar una función para ingresar a través del teclado un número natural. La función rechazará cualquier ingreso inválido de datos utilizando excepciones y mostrará la razón exacta del error. Controlar que se ingrese un número, que ese número sea entero y que sea mayor que 0, mostrando un mensaje con la razón exacta del error en caso necesario. Devolver el valor ingresado cuando éste sea correcto. Escribir también un programa que permita probar el correcto funcionamiento de la misma.
2. Realizar una función que reciba como parámetros dos cadenas de caracteres conteniendo números reales, sume ambos valores y devuelva el resultado como un número real. Devolver -1 si alguna de las cadenas no contiene un número válido, utilizando manejo de excepciones para detectar el error.
3. Desarrollar una función que devuelva una cadena de caracteres con el nombre del mes cuyo número se recibe como parámetro. Los nombres de los meses deberán obtenerse de una lista de cadenas de caracteres inicializada dentro de la función. Devolver una cadena vacía si el número de mes es inválido. La detección de meses inválidos deberá realizarse a través de excepciones.
4. Todo programa Python es susceptible de ser interrumpido mediante la pulsación de las teclas Ctrl-C, lo que genera una excepción del tipo `KeyboardInterrupt`. Realizar un programa para imprimir los números enteros entre 1 y 100000, y que solicite confirmación al usuario antes de detenerse cuando se presione Ctrl-C.
5. La raíz cuadrada de un número puede obtenerse mediante la función `sqrt()` del módulo `math`. Escribir un programa que utilice esta función para calcular la raíz cuadrada de un número cualquiera ingresado a través del teclado. El programa debe utilizar manejo de excepciones para evitar errores si se ingresa un número negativo.
6. El método `index` permite buscar un elemento dentro de una lista, devolviendo la posición que éste ocupa. Sin embargo, si el elemento no pertenece a la lista se produce una excepción de tipo `ValueError`. Desarrollar un programa que cargue una lista con números enteros ingresados a través del teclado (terminando con -1) y permita que el usuario ingrese el valor de algunos elementos para visualizar la posición que ocupan, utilizando el método `index`. Si el número no pertenece a la lista se imprimirá un mensaje de error y se solicitará otro para buscar. Abortar el proceso al tercer error detectado. No utilizar el operador `in` durante la búsqueda.
7. Escribir un programa que juegue con el usuario a adivinar un número. El programa debe generar un número al azar entre 1 y 500 y el usuario debe adivinarlo. Para eso, cada vez que se introduce un valor se muestra un mensaje indicando si el número que tiene que adivinar es mayor o menor que el ingresado. Cuando consiga adivinarlo, se debe imprimir en pantalla la cantidad de intentos que le tomó hallar el número. Si el usuario introduce algo que no sea un número se mostrará un mensaje en pantalla y se lo contará como un intento más.

Trabajo Práctico 7: Archivos

1. Escribir un programa que lea un archivo de texto conteniendo un conjunto de apellidos y nombres en formato "Apellido, Nombre" y guarde en el archivo ARMENIA.TXT los registros de aquellas personas cuyo apellido termina con la cadena "IAN", en el archivo ITALIA.TXT los terminados en "INI" y en ESPAÑA.TXT los terminados en "EZ". Descartar el resto. Ejemplo:

Arslanian, Gustavo	-> ARMENIA.TXT
Rossini, Giuseppe	-> ITALIA.TXT
Pérez, Juan	-> ESPAÑA.TXT
Smith, John	-> descartar

El archivo puede ser creado mediante el Block de Notas o el cualquier otro editor.

2. Escribir un programa que permita dividir un archivo de texto cualquiera en partes que se puedan enviar por correo electrónico. El tamaño máximo de las partes se ingresa por teclado. Los nombres de los archivos generados deben respetar el nombre original con el agregado de un sufijo que indique de qué parte se trata. Tener en cuenta que ningún registro puede ser dividido; la partición debe efectuarse después del delimitador del mismo. Mostrar un mensaje de error si el proceso no pudiera llevarse a cabo. Recordar que no se permite cargar el archivo completo en memoria.
3. Una institución deportiva necesita clasificar a sus atletas para inscribirlos en los próximos Juegos Panamericanos. Para eso encargó la realización de un programa que incluya las siguientes funciones:

GrabarRangoAlturas(): Graba en un archivo las alturas de los atletas de distintas disciplinas, los que se ingresan desde el teclado. Cada dato se debe grabar en una línea distinta. Ejemplo:

```
<Deporte 1>
<altura del atleta 1>
<altura del atleta 2>
< . . . >
<Deporte 2>
<altura del atleta 1>
<altura del atleta 2>
< . . . >
```

GrabarPromedio(): Graba en un archivo los promedios de las alturas de los atletas, leyendo los datos del archivo generado en el paso anterior. La disciplina y el promedio deben grabarse en líneas diferentes. Ejemplo:

```
<Deporte 1>
<Promedio de alturas deporte 1>
<Deporte 2>
<Promedio de alturas deporte 2>
< . . . >
```

MostrarMasAltos() Muestra por pantalla las disciplinas deportivas cuyos atletas superan la estatura promedio general. Obtener los datos del segundo archivo.

4. Desarrollar un programa para eliminar todos los comentarios de un programa escrito en lenguaje Python. Tener en cuenta que los comentarios comienzan con el signo # (siempre que éste no se encuentre encerrado entre comillas simples o dobles) y que también se considera comentario a las cadenas de documentación (docstrings).
5. Se dispone de dos formatos diferentes de archivos de texto en los que se almacenan datos de empleados, detallados más abajo. Desarrollar un programa para convertir cada uno de los formatos suministrados, grabando los datos obtenidos en otro archivo con formato CSV. Los archivos de entrada pueden generarse con Block de Notas o cualquier otro editor, copiando y pegando los ejemplos proporcionados. Ambos archivos tienen tres campos por registro: Apellido y Nombre, Fecha de alta y Domicilio.

Formato 1: Los campos tienen longitud fija con un espacio de separación entre ellos.

(Regla)	1	2	3	4	5	6
	012345678901234567890123456789012345678901234567890123456789012					

Pérez Juan	20080211	Corrientes 348
González Ana M	20080115	Juan de Garay 1111 3er piso dto A

Formato 2: Todos los campos de este archivo están precedidos por un número de dos dígitos que indica la longitud del campo que sigue.

10	Pérez Juan	08	20080211	14	Corrientes 348
14	González Ana M	08	20080115	33	Juan de Garay 1111 3er piso dto A

NOTAS:

- Los espacios que se encuentren al final de las cadenas en el formato 1 deberán ser eliminados.
- El formato 2 debe generalizarse para que soporte registros con cualquier cantidad de campos.

6. Un hotel necesita un programa para gestionar la operación de sus habitaciones. El hotel cuenta con 10 pisos y 6 habitaciones por piso. Por cada huésped o grupo familiar que se aloja en el mismo se registra la siguiente información:

- DNI del cliente (número entero)
- Apellido y Nombre
- Fecha de ingreso (DDMMAAAA)
- Fecha de egreso (DDMMAAAA)
- Cantidad de ocupantes

Se solicita desarrollar un programa para realizar las siguientes tareas:

- Registrar el ingreso de huéspedes al hotel, hasta que se ingrese un número de DNI -1. Esta información deberá grabarse en un archivo CSV donde cada registro incluirá todos los campos indicados más arriba. Tener en cuenta que los números de DNI no pueden repetirse y que la fecha de salida debe ser mayor a la de entrada.

- Finalizado el ingreso de huéspedes se solicita:
 - a. Leer el archivo de huéspedes y asignar la habitaciones a cada uno. El piso y habitación son asignados arbitrariamente, y no puede asignarse una habitación ya otorgada.
 - b. Mostrar el piso con mayor cantidad de habitaciones ocupadas.
 - c. Mostrar cuántas habitaciones vacías hay en todo el hotel.
 - d. Mostrar el piso con mayor cantidad de personas.
 - e. Mostrar cuál será la próxima habitación en desocuparse. La fecha actual se ingresa por teclado. Mostrar todas las que correspondan.
 - f. Mostrar un listado de todos los huéspedes registrados en el hotel, ordenado por cantidad de días de alojamiento.

Trabajo Práctico 8: Recursividad

1. Escribir una función recursiva que devuelva la cantidad de dígitos de un número entero, sin utilizar cadenas de caracteres ni otras estructuras de datos.
2. Desarrollar una función recursiva que reciba un número binario y lo devuelva convertido a base decimal.
3. Escribir una función recursiva que devuelva la suma de los N primeros números naturales.
4. Desarrollar una función recursiva que devuelva el producto de dos números enteros por sumas sucesivas.
5. Realizar una función recursiva que devuelva el resto de dos números enteros, utilizando restas sucesivas.
6. La función de Ackermann $A(m,n)$ se define de la siguiente forma:

$$\begin{array}{ll} n+1 & \text{si } m = 0 \\ A(m-1,1) & \text{si } n = 0 \\ A(m-1, A(m,n-1)) & \text{de otro modo} \end{array}$$

Imprimir un cuadro con los valores que adopta la función para valores de m entre 0 y 3 y de n entre 0 y 7.

7. Dados dos números enteros no negativos, devolver el resultado de calcular el Máximo Común Divisor (también llamado Divisor Común Mayor) basándose en las siguientes propiedades:

$$\begin{array}{ll} \text{MCD}(X, X) = & X \\ \text{MCD}(X, Y) = & \text{MCD}(Y, X) \\ \text{Si } X > Y \Rightarrow & \text{MCD}(X, Y) = \text{MCD}(X-Y, Y). \end{array}$$

Utilizando la función anterior calcular el MCD de todos los elementos de una lista de números enteros, sabiendo que $\text{MCD}(X,Y,Z) = \text{MCD}(\text{MCD}(X,Y),Z)$. No se permite utilizar iteraciones en ninguna etapa del proceso.

8. Realizar la implementación recursiva del método de selección para ordenar una lista de números enteros. Sugerencia: Colocar el elemento más pequeño en la primera posición, y luego ordenar el resto de la lista con una llamada recursiva. No usar las funciones `min()` ni `max()` de Python.
9. Realizar una función recursiva para imprimir una matriz de M x N con el formato apropiado.
10. Escribir una función recursiva que sume todos los elementos de una matriz de M x N y devuelva el resultado. No usar la función `sum()`.
11. Desarrollar una función que devuelva el mínimo elemento de una matriz de M x N.